

CSCI 331 Project 3

Generated by Doxygen 1.16.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 BFieldDescriptor Struct Reference	5
3.1.1 Detailed Description	5
3.2 BlockBuffer Class Reference	5
3.2.1 Detailed Description	6
3.2.2 Constructor & Destructor Documentation	7
3.2.2.1 BlockBuffer()	7
3.2.3 Member Function Documentation	7
3.2.3.1 addRecord()	7
3.2.3.2 clear()	8
3.2.3.3 freeBytes()	8
3.2.3.4 getRecords()	8
3.2.3.5 highestKey()	9
3.2.3.6 lowestKey()	9
3.2.3.7 makeAvail()	9
3.2.3.8 offsetForRBN()	10
3.2.3.9 pack()	10
3.2.3.10 readBlock()	11
3.2.3.11 unpack()	11
3.2.3.12 writeBlock()	12
3.3 BlockFileHeader Struct Reference	13
3.3.1 Detailed Description	13
3.4 BlockHeaderBuffer Class Reference	14
3.4.1 Detailed Description	14
3.4.2 Constructor & Destructor Documentation	14
3.4.2.1 BlockHeaderBuffer()	14
3.4.3 Member Function Documentation	15
3.4.3.1 buildDefault()	15
3.4.3.2 print()	15
3.4.3.3 read()	15
3.4.3.4 write()	16
3.5 SequenceSet Class Reference	16
3.5.1 Detailed Description	17
3.5.2 Constructor & Destructor Documentation	17
3.5.2.1 SequenceSet()	17
3.5.3 Member Function Documentation	18
3.5.3.1 close()	18

3.5.3.2 create()	18
3.5.3.3 dumpIndex()	19
3.5.3.4 dumpLogical()	19
3.5.3.5 dumpPhysical()	19
3.5.3.6 insert()	19
3.5.3.7 isOpen()	20
3.5.3.8 open()	20
3.5.3.9 printHeader()	21
3.5.3.10 remove()	21
3.5.3.11 scanAll()	22
3.5.3.12 search()	22
3.6 SimpleIndex Class Reference	23
3.6.1 Detailed Description	23
3.6.2 Member Function Documentation	23
3.6.2.1 entries()	23
3.6.2.2 findRBN()	24
3.6.2.3 read()	24
3.6.2.4 removeByKey()	25
3.6.2.5 removeByRBN()	25
3.6.2.6 size()	26
3.6.2.7 upsert()	26
3.6.2.8 write()	26
3.7 StateExtremes Struct Reference	27
3.7.1 Detailed Description	27
3.8 ZipCodeBuffer Class Reference	27
3.8.1 Detailed Description	28
3.8.2 Constructor & Destructor Documentation	28
3.8.2.1 ZipCodeBuffer()	28
3.8.3 Member Function Documentation	28
3.8.3.1 gatherAllRecords()	28
3.8.3.2 open()	29
3.8.3.3 readRecord()	29
3.8.3.4 reset()	29
3.9 ZipCodeRecord Struct Reference	30
3.9.1 Detailed Description	30
3.9.2 Member Function Documentation	31
3.9.2.1 fromCSV()	31
3.9.2.2 toCSV()	31
4 File Documentation	33
4.1 BlockBuffer.cpp File Reference	33
4.1.1 Detailed Description	33

4.2 BlockBuffer.h File Reference	33
4.2.1 Detailed Description	34
4.3 BlockBuffer.h	34
4.4 BlockHeaderBuffer.cpp File Reference	35
4.4.1 Detailed Description	36
4.5 BlockHeaderBuffer.h File Reference	36
4.5.1 Detailed Description	36
4.6 BlockHeaderBuffer.h	37
4.7 main.cpp File Reference	38
4.7.1 Detailed Description	39
4.8 SequenceSet.cpp File Reference	40
4.8.1 Detailed Description	40
4.9 SequenceSet.h File Reference	40
4.9.1 Detailed Description	41
4.10 SequenceSet.h	41
4.11 SimpleIndex.cpp File Reference	43
4.11.1 Detailed Description	43
4.12 SimpleIndex.h File Reference	43
4.12.1 Detailed Description	43
4.13 SimpleIndex.h	44
4.14 ZipCodeBuffer.cpp File Reference	44
4.14.1 Detailed Description	44
4.15 ZipCodeBuffer.h File Reference	45
4.15.1 Detailed Description	45
4.16 ZipCodeBuffer.h	45

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BFieldDescriptor	Stores the name and type format for one field in the blocked file	5
BlockBuffer	Reads and writes fixed-size blocks in a blocked sequence set file	5
BlockFileHeader	All header fields for a blocked sequence set data file	13
BlockHeaderBuffer	Reads and writes the header record for a blocked sequence set file	14
SequenceSet	Manages a blocked sequence set file for ZIP code records	16
SimpleIndex	In-RAM representation of the simple primary key index	23
StateExtremes	Keeps the most extreme ZIP codes for one state	27
ZipCodeBuffer	Reads ZIP code records one at a time from a CSV file	27
ZipCodeRecord	Stores one complete ZIP code record	30

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

BlockBuffer.cpp	Implementation of the BlockBuffer class	33
BlockBuffer.h	Block buffer class for the blocked sequence set file	33
BlockHeaderBuffer.cpp	Implementation of BlockHeaderBuffer	35
BlockHeaderBuffer.h	Header record class for the blocked sequence set data file	36
main.cpp	Zip Code Group Project 3.0 — Blocked Sequence Set main controller	38
SequenceSet.cpp	Implementation of the SequenceSet class	40
SequenceSet.h	High-level manager for the blocked sequence set file	40
SimpleIndex.cpp	Implementation of SimpleIndex	43
SimpleIndex.h	Simple primary key index for the blocked sequence set	43
ZipCodeBuffer.cpp	Implementation of the ZipCodeBuffer class	44
ZipCodeBuffer.h	Record buffer class for reading ZIP code records from CSV files	45

Chapter 3

Class Documentation

3.1 BFieldDescriptor Struct Reference

Stores the name and type format for one field in the blocked file.

```
#include <BlockHeaderBuffer.h>
```

Public Attributes

- string **name**
e.g. "ZipCode"
- string **format**
e.g. "int", "string", "double"

3.1.1 Detailed Description

Stores the name and type format for one field in the blocked file.

The documentation for this struct was generated from the following file:

- [BlockHeaderBuffer.h](#)

3.2 BlockBuffer Class Reference

Reads and writes fixed-size blocks in a blocked sequence set file.

```
#include <BlockBuffer.h>
```

Public Member Functions

- [BlockBuffer](#) (int blockSize=DEFAULT_BLOCK_SIZE)
Construct a [BlockBuffer](#) for the given block size.
- int **getBlockSize** () const
Return the configured block size in bytes.
- int **getRecordCount** () const
Return number of records currently in buffer.
- int **getPredRBN** () const
Predecessor block RBN (RBN_NULL if none).
- int **getSuccRBN** () const
Successor block RBN (RBN_NULL if none).
- bool **isAvail** () const
True if this is an avail-list block (count==0).
- void **setPredRBN** (int rbn)
Set predecessor link.
- void **setSuccRBN** (int rbn)
Set successor link.
- const vector< string > & **getRecords** () const
Get all record strings currently in the buffer.
- bool **addRecord** (const string &csvRecord)
Add a record string to the buffer.
- int **freeBytes** () const
How many bytes are still free in the block (excluding metadata).
- void **clear** ()
Clear all records from the buffer (keeps metadata/blockSize).
- string **pack** () const
Pack the buffer into a fixed-size byte array ready to write.
- bool **unpack** (const string &raw)
Unpack a fixed-size block from a raw byte string.
- bool **writeBlock** (fstream &fs, int rbn) const
Write this buffer to a specific RBN in the file.
- bool **readBlock** (fstream &fs, int rbn)
Read a specific RBN from the file into this buffer.
- long long **offsetForRBN** (int rbn) const
Compute the file byte offset for a given RBN.
- void **makeAvail** (int nextAvail=RBN_NULL)
Mark this buffer as an avail-list block.
- string **highestKey** () const
Return the highest (largest) key string currently in the buffer.
- string **lowestKey** () const
Return the lowest key string currently in the buffer.

3.2.1 Detailed Description

Reads and writes fixed-size blocks in a blocked sequence set file.

Responsibilities:

- Pack a set of CSV record strings + metadata into a fixed-size block buffer.

- Unpack records from a block buffer back into strings.
- Read/write a block from/to disk by Relative Block Number (RBN).
- Manage per-block metadata: predecessor RBN, successor RBN, record count.

This class does NOT know about ZIP codes specifically — it works with arbitrary CSV strings. The caller ([ZipCodeRecord](#) / main) converts strings to typed objects.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 BlockBuffer()

```
BlockBuffer::BlockBuffer (  
    int blockSize = DEFAULT_BLOCK_SIZE) [explicit]
```

Construct a [BlockBuffer](#) for the given block size.

Construct a [BlockBuffer](#) with the given block size.

Parameters

<i>blockSize</i>	Total bytes per block (default DEFAULT_BLOCK_SIZE).
------------------	---

All links default to RBN_NULL; the record list is empty.

Parameters

<i>blockSize</i>	Bytes per block.
------------------	------------------

3.2.3 Member Function Documentation

3.2.3.1 addRecord()

```
bool BlockBuffer::addRecord (  
    const string & csvRecord)
```

Add a record string to the buffer.

Attempt to add a CSV record string to this buffer.

Returns false (and does not add) if there is not enough remaining capacity for the record + its 4-byte length prefix.

Parameters

<i>csvRecord</i>	CSV text for one record (no newline).
------------------	---------------------------------------

Returns

true if added, false if block is full.

Each record occupies RECORD_LEN_WIDTH bytes for its length prefix plus the length of the record text itself.

Parameters

<i>csvRecord</i>	CSV text (no newline).
------------------	------------------------

Returns

true if the record fit, false if the block is full.

3.2.3.2 clear()

```
void BlockBuffer::clear ()
```

Clear all records from the buffer (keeps metadata/blockSize).

Clear all records from the buffer.

3.2.3.3 freeBytes()

```
int BlockBuffer::freeBytes () const
```

How many bytes are still free in the block (excluding metadata).

Return the number of free bytes in the block payload.

Returns

Free bytes available for additional records.

3.2.3.4 getRecords()

```
const vector< string > & BlockBuffer::getRecords () const
```

Get all record strings currently in the buffer.

Returns

Const reference to the internal record vector.

3.2.3.5 highestKey()

```
string BlockBuffer::highestKey () const
```

Return the highest (largest) key string currently in the buffer.

Return the highest (last, largest) key in the buffer.

Keys are the first CSV field (ZIP code). Assumes records are sorted. Returns "" if buffer is empty.

The key is the ZIP code — the first field of the CSV record. Assumes records_ is sorted ascending by key.

Returns

Highest key string, or "" if the buffer is empty.

3.2.3.6 lowestKey()

```
string BlockBuffer::lowestKey () const
```

Return the lowest key string currently in the buffer.

Return the lowest (first, smallest) key in the buffer.

Returns "" if buffer is empty.

Returns

Lowest key string, or "" if the buffer is empty.

3.2.3.7 makeAvail()

```
void BlockBuffer::makeAvail (
    int nextAvail = RBN_NULL)
```

Mark this buffer as an avail-list block.

Configure this buffer as an avail-list block.

Sets record count to 0, predecessor to RBN_NULL, fills payload with spaces. The successor RBN is set to nextAvail (RBN_NULL means end-of-list).

Parameters

<i>nextAvail</i>	RBN of next avail block, or RBN_NULL.
------------------	---------------------------------------

An avail block has record count 0 and payload filled with spaces. The successor RBN links to the next avail block (or RBN_NULL).

Parameters

<i>nextAvail</i>	RBN of the next avail block, or RBN_NULL for end-of-list.
------------------	---

3.2.3.8 offsetForRBN()

```
long long BlockBuffer::offsetForRBN (
    int rbn) const
```

Compute the file byte offset for a given RBN.

Offset = rbn * blockSize. RBN 0 is reserved for the file header (also exactly one block).

Parameters

<i>rbn</i>	Relative Block Number.
------------	------------------------

Returns

Byte offset from start of file.

Parameters

<i>rbn</i>	Relative Block Number.
------------	------------------------

Returns

Byte offset from the start of the file.

3.2.3.9 pack()

```
string BlockBuffer::pack () const
```

Pack the buffer into a fixed-size byte array ready to write.

Pack the buffer into a fixed-size string of blockSize_ bytes.

Layout: [4-byte count][8-byte predRBN][8-byte succRBN] [4-byte len][record][4-byte len][record]... [padding with spaces to blockSize]

Returns

String of exactly blockSize bytes.

Layout: [4-byte count][8-byte predRBN][8-byte succRBN] [4-byte len][record text][4-byte len][record text]... [padding spaces]

RBN_NULL is stored as "FFFFFFFF".

Returns

String of exactly blockSize_ bytes.

3.2.3.10 readBlock()

```
bool BlockBuffer::readBlock (
    fstream & fs,
    int rbn)
```

Read a specific RBN from the file into this buffer.

Read a block from disk by RBN into this buffer.

Seeks to the correct byte offset and reads exactly blockSize bytes, then calls [unpack\(\)](#).

Parameters

<i>fs</i>	Open fstream.
<i>rbn</i>	Relative Block Number to read.

Returns

true on success.

Parameters

<i>fs</i>	Open fstream.
<i>rbn</i>	Relative Block Number.

Returns

true on success.

3.2.3.11 unpack()

```
bool BlockBuffer::unpack (
    const string & raw)
```

Unpack a fixed-size block from a raw byte string.

Unpack a raw block byte string into this buffer.

Reads the metadata header and all length-prefixed record strings.

Parameters

<i>raw</i>	Raw byte string of exactly blockSize bytes.
------------	---

Returns

true on success, false on format error.

Reads the metadata header and all length-prefixed records.

Parameters

<i>raw</i>	Raw byte string of exactly blockSize_bytes.
------------	---

Returns

true on success, false on format/length error.

3.2.3.12 writeBlock()

```
bool BlockBuffer::writeBlock (
    fstream & fs,
    int rbn) const
```

Write this buffer to a specific RBN in the file.

Write this buffer to disk at the position for rbn.

The file must be open for read/write (fstream). Seeks to the correct byte offset and writes exactly blockSize bytes.

Parameters

<i>fs</i>	Open fstream (binary or text, seekable).
<i>rbn</i>	Relative Block Number to write (RBN 0 = header block).

Returns

true on success.

Parameters

<i>fs</i>	Open fstream (must be seekable, read+write).
<i>rbn</i>	Relative Block Number.

Returns

true on success.

The documentation for this class was generated from the following files:

- [BlockBuffer.h](#)
- [BlockBuffer.cpp](#)

3.3 BlockFileHeader Struct Reference

All header fields for a blocked sequence set data file.

```
#include <BlockHeaderBuffer.h>
```

Public Attributes

- string **fileType**
"ZipBlockedSeqSet"
- int **version**
File structure version (start at 1).
- int **headerSizeBytes**
Byte length of the serialized header record.
- int **recordSizeBytes**
Width of per-record length prefix (4 bytes).
- string **sizeFormatType**
"ASCII"
- int **blockSize**
Bytes per block (default 512).
- int **minBlockCapacityPct**
Minimum fill percentage (default 50).
- string **indexFileName**
Name of the simple index file (.idx).
- string **indexSchema**
Describes index format, e.g. "key:string;rbn:int".
- long long **recordCount**
Total ZIP records stored.
- int **blockCount**
Total blocks allocated (including header block).
- int **fieldCount**
Number of fields per record.
- vector< [BFieldDescriptor](#) > **fields**
One entry per field.
- int **primaryKeyIndex**
0-based field index of the primary key
- int **availHeadRBN**
Head of avail-list chain (RBN_NULL if empty).
- int **seqSetHeadRBN**
Head of active sequence set chain (first data block).
- bool **staleFlag**
True if index may be out of date.

3.3.1 Detailed Description

All header fields for a blocked sequence set data file.

The documentation for this struct was generated from the following file:

- [BlockHeaderBuffer.h](#)

3.4 BlockHeaderBuffer Class Reference

Reads and writes the header record for a blocked sequence set file.

```
#include <BlockHeaderBuffer.h>
```

Public Member Functions

- [BlockHeaderBuffer](#) (int blockSize=512)
Construct with a given block size.
- void [buildDefault](#) (const string &indexFileName, int blockSize, long long recordCount, int blockCount, int availHead, int seqHead)
Populate the header with default values for the ZIP code file.
- bool [write](#) (fstream &fs) const
Write the header to RBN 0 of an open fstream.
- bool [read](#) (fstream &fs)
Read the header from RBN 0 of an open fstream.
- const [BlockFileHeader](#) & [getHeader](#) () const
- [BlockFileHeader](#) & [getHeader](#) ()
- void [setRecordCount](#) (long long count)
- void [setBlockCount](#) (int count)
- void [setAvailHead](#) (int rbn)
- void [setSeqSetHead](#) (int rbn)
- void [setStale](#) (bool s)
- void [print](#) () const
Print the header contents to cout.

3.4.1 Detailed Description

Reads and writes the header record for a blocked sequence set file.

The header occupies exactly one block (RBN 0). It is stored as a length-indicated CSV record padded with spaces to blockSize bytes.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 BlockHeaderBuffer()

```
BlockHeaderBuffer::BlockHeaderBuffer (
    int blockSize = 512) [explicit]
```

Construct with a given block size.

Construct a [BlockHeaderBuffer](#).

Parameters

<i>blockSize</i>	Block size in bytes (must match the data file).
------------------	---

3.4.3 Member Function Documentation

3.4.3.1 buildDefault()

```
void BlockHeaderBuffer::buildDefault (
    const string & indexFileName,
    int blockSize,
    long long recordCount,
    int blockCount,
    int availHead,
    int seqHead)
```

Populate the header with default values for the ZIP code file.

Populate the header with default values for the ZIP code blocked file.

Call this before writing a brand-new blocked sequence set file.

Parameters

<i>indexFileName</i>	Name of the simple primary key index file.
<i>blockSize</i>	Block size in bytes.
<i>recordCount</i>	Total ZIP records to be stored (may be 0 initially).
<i>blockCount</i>	Total blocks allocated (including RBN 0).
<i>availHead</i>	RBN of first avail block (RBN_NULL if none).
<i>seqHead</i>	RBN of first active data block.

3.4.3.2 print()

```
void BlockHeaderBuffer::print () const
```

Print the header contents to cout.

Print the header contents to cout in a readable format.

3.4.3.3 read()

```
bool BlockHeaderBuffer::read (
    fstream & fs)
```

Read the header from RBN 0 of an open fstream.

Read the header record from RBN 0 of the open fstream.

Parameters

<i>fs</i>	Open fstream.
-----------	---------------

Returns

true on success.

3.4.3.4 write()

```
bool BlockHeaderBuffer::write (
    fstream & fs) const
```

Write the header to RBN 0 of an open fstream.

Write the header record to RBN 0 of the open fstream.

The header record is written as: [10-byte ASCII length][space][CSV text][padding to blockSize]

Parameters

<i>fs</i>	Open fstream positioned at any location (seeked internally).
-----------	--

Returns

true on success.

Format written at byte offset 0: [10-byte ASCII length][space][CSV text][spaces to blockSize_]

Parameters

<i>fs</i>	Open fstream (read+write, seekable).
-----------	--------------------------------------

Returns

true on success.

The documentation for this class was generated from the following files:

- [BlockHeaderBuffer.h](#)
- [BlockHeaderBuffer.cpp](#)

3.5 SequenceSet Class Reference

Manages a blocked sequence set file for ZIP code records.

```
#include <SequenceSet.h>
```

Public Member Functions

- [SequenceSet](#) (int blockSize=DEFAULT_BLOCK_SIZE)
Construct a [SequenceSet](#) manager.
- [~SequenceSet](#) ()
Destructor — close the file if it is still open.
- bool [create](#) (const string &csvFile, const string &dataFile, const string &indexFile)
Create a brand-new blocked sequence set from a sorted CSV file.
- bool [open](#) (const string &dataFile, const string &indexFile)
Open an existing blocked sequence set (data + index).
- void [close](#) ()
Close the data file and flush the header.
- bool [isOpen](#) () const
Check whether the file is open.
- void [scanAll](#) (function< void(const [ZipCodeRecord](#) &)> visitor) const
Read all records in logical order and call visitor for each.
- void [dumpPhysical](#) () const
Dump blocks in physical order (by RBN 1, 2, 3, ...).
- void [dumpLogical](#) () const
Dump blocks in logical order (following successor chain).
- void [dumpIndex](#) () const
Print a readable dump of the simple index.
- bool [search](#) (const string &zipKey, [ZipCodeRecord](#) &result)
Search for a ZIP code record by key.
- bool [insert](#) (const [ZipCodeRecord](#) &rec)
Insert a new ZIP code record into the sequence set.
- bool [remove](#) (const string &zipKey)
Delete a ZIP code record from the sequence set.
- void [printHeader](#) () const
Print the file header contents to cout.

3.5.1 Detailed Description

Manages a blocked sequence set file for ZIP code records.

3.5.2 Constructor & Destructor Documentation

3.5.2.1 SequenceSet()

```
SequenceSet::SequenceSet (
    int blockSize = DEFAULT_BLOCK_SIZE) [explicit]
```

Construct a [SequenceSet](#) manager.

Construct a [SequenceSet](#) manager with the given block size.

Parameters

<i>blockSize</i>	Block size in bytes (default 512).
------------------	------------------------------------

3.5.3 Member Function Documentation**3.5.3.1 close()**

```
void SequenceSet::close ()
```

Close the data file and flush the header.

Close the data file, flushing the header first.

3.5.3.2 create()

```
bool SequenceSet::create (
    const string & csvFile,
    const string & dataFile,
    const string & indexFile)
```

Create a brand-new blocked sequence set from a sorted CSV file.

Steps: 1) Read the CSV one record at a time (streaming). 2) Pack records into blocks until each block hits 100% capacity. 3) Write each completed block to disk. 4) Build the [SimpleIndex](#) entry for each block. 5) Write the index file. 6) Write the file header (RBN 0).

The CSV must already be sorted ascending by ZIP code.

Parameters

<i>csvFile</i>	Path to sorted CSV file.
<i>dataFile</i>	Path of the blocked sequence set to create.
<i>indexFile</i>	Path of the simple index file to create.

Returns

true on success.

The CSV must already be sorted ascending by ZIP code (first field). Each block is filled to capacity before a new block is started.

Parameters

<i>csvFile</i>	Sorted CSV input.
<i>dataFile</i>	Output blocked sequence set file.
<i>indexFile</i>	Output simple index file.

Returns

true on success.

3.5.3.3 dumpIndex()

```
void SequenceSet::dumpIndex () const
```

Print a readable dump of the simple index.

Print a readable dump of the simple index to cout.

3.5.3.4 dumpLogical()

```
void SequenceSet::dumpLogical () const
```

Dump blocks in logical order (following successor chain).

Dump blocks in logical (successor chain) order.

Same format as dumpPhysical but blocks appear in linked-list order.

3.5.3.5 dumpPhysical()

```
void SequenceSet::dumpPhysical () const
```

Dump blocks in physical order (by RBN 1, 2, 3, ...).

Dump all blocks in physical (RBN) order.

Format: List Head: <rbn> Avail Head: <rbn> <predRBN> key1 key2 ... keyN <succRBN> <predRBN> *available* <succRBN> ...

3.5.3.6 insert()

```
bool SequenceSet::insert (
    const ZipCodeRecord & rec)
```

Insert a new ZIP code record into the sequence set.

Insert a new record into the sequence set.

Algorithm: 1) Use the index to find the target block. 2) If the record fits, insert in sorted position. 3) If the block is full, split it: a) Obtain an avail block (from avail list or append new block). b) Distribute records evenly between the two blocks. c) Update predecessor/successor links. d) Update the index. Logs block splits to cout.

Parameters

<i>rec</i>	Record to insert.
------------	-------------------

Returns

true on success, false if the record already exists.

Finds the target block using the index, inserts the record in sorted order, and splits the block if it is full.

Parameters

<i>rec</i>	Record to insert.
------------	-------------------

Returns

true on success, false if the key already exists.

3.5.3.7 isOpen()

```
bool SequenceSet::isOpen () const
```

Check whether the file is open.

Return whether the file is currently open.

3.5.3.8 open()

```
bool SequenceSet::open (
    const string & dataFile,
    const string & indexFile)
```

Open an existing blocked sequence set (data + index).

Open an existing blocked sequence set.

Reads the header from RBN 0, then loads the index into RAM.

Parameters

<i>dataFile</i>	Path to the blocked sequence set file.
<i>indexFile</i>	Path to the simple index file.

Returns

true on success.

Reads the header (RBN 0) and loads the index into RAM.

Parameters

<i>dataFile</i>	Path to the blocked sequence set file.
-----------------	--

<i>indexFile</i>	Path to the simple index file.
------------------	--------------------------------

Returns

true on success.

3.5.3.9 printHeader()

```
void SequenceSet::printHeader () const
```

Print the file header contents to cout.

Print the file header to cout.

3.5.3.10 remove()

```
bool SequenceSet::remove (  
    const string & zipKey)
```

Delete a ZIP code record from the sequence set.

Delete a record by ZIP key.

Algorithm: 1) Use the index to find the target block. 2) Remove the record from the block. 3) If the block is below the minimum fill: a) Try redistribution with the left or right neighbour. b) If redistribution is not possible, merge with a neighbour. The merged block is added to the avail list. Logs merges / redistributions to cout.

Parameters

<i>zipKey</i>	ZIP code string to delete.
---------------	----------------------------

Returns

true if found and deleted, false if not found.

After deletion, if the block falls below minimum fill, attempts redistribution with a neighbour; if that is not possible, merges with a neighbour.

Parameters

<i>zipKey</i>	ZIP code string to delete.
---------------	----------------------------

Returns

true if found and deleted.

3.5.3.11 scanAll()

```
void SequenceSet::scanAll (
    function< void(const ZipCodeRecord &)> visitor) const
```

Read all records in logical order and call visitor for each.

Visit every record in logical (key) order.

Follows the successor RBN chain from seqSetHead. Each block is unpacked; each record string is parsed into a [ZipCodeRecord](#).

Parameters

<i>visitor</i>	Callback called with each ZipCodeRecord in key order.
----------------	---

Follows the successor chain starting from seqSetHeadRBN. For each active block, reads its records and calls visitor.

Parameters

<i>visitor</i>	Callback receiving each ZipCodeRecord .
----------------	---

3.5.3.12 search()

```
bool SequenceSet::search (
    const string & zipKey,
    ZipCodeRecord & result)
```

Search for a ZIP code record by key.

Search for a record by ZIP key.

Uses the index to find the candidate block, reads that block from disk, and performs a linear search within the block.

Parameters

<i>zipKey</i>	ZIP code as a string (e.g. "56301").
<i>result</i>	Output record if found.

Returns

true if found, false if not in the file.

Uses the simple index to locate the candidate block, then searches that block linearly.

Parameters

<i>zipKey</i>	ZIP code string (e.g. "56301").
<i>result</i>	Output ZipCodeRecord if found.

Returns

true if found.

The documentation for this class was generated from the following files:

- [SequenceSet.h](#)
- [SequenceSet.cpp](#)

3.6 SimpleIndex Class Reference

In-RAM representation of the simple primary key index.

```
#include <SimpleIndex.h>
```

Public Member Functions

- **SimpleIndex** ()
Construct an empty [SimpleIndex](#).
- void **upsert** (const string &highestKey, int rbn)
Insert or update an entry {highestKey → rbn}.
- void **removeByRBN** (int rbn)
Remove the entry whose value equals rbn.
- void **removeByKey** (const string &highestKey)
Remove the entry whose key equals highestKey.
- int **findRBN** (const string &targetKey) const
Find the RBN of the block that could contain targetKey.
- bool **write** (const string &filename) const
Write the index to a text file.
- bool **read** (const string &filename)
Load the index from a text file into RAM.
- int **size** () const
Return the number of entries in the index.
- vector< pair< string, int > > **entries** () const
Return all entries as {key, rbn} pairs in key order.
- void **dump** () const
Print a readable dump of the index to cout.

3.6.1 Detailed Description

In-RAM representation of the simple primary key index.

The index maps each active block's highest key to its RBN. It uses a std::map so the entries are always sorted by key.

Usage pattern: 1) Build the index during file creation (insert entries for each block). 2) Write to disk. 3) At search time, load from disk into RAM. 4) To find a ZIP code, call [findRBN\(\)](#); it returns the RBN of the block whose highest key >= the target key (the block that could contain it).

3.6.2 Member Function Documentation

3.6.2.1 entries()

```
vector< pair< string, int > > SimpleIndex::entries () const
```

Return all entries as {key, rbn} pairs in key order.

Return all entries as a sorted vector of {key, rbn} pairs.

3.6.2.2 findRBN()

```
int SimpleIndex::findRBN (
    const string & targetKey) const
```

Find the RBN of the block that could contain targetKey.

Looks up the first block whose highest key $\geq$ targetKey. If no such block exists, returns -1 (the key is beyond the last block).

Parameters

<i>targetKey</i>	ZIP code string to search for (e.g. "56301").
------------------	---

Returns

RBN of the candidate block, or -1 if not found.

Uses lower_bound to find the first entry whose key $\geq$ targetKey. That block is the only one that could contain the record.

Example: Index: 01005→1, 10199→2, 56310→3 Search for "05678": lower_bound("05678") → key "10199" → RBN 2 Block 2's range is (01005, 10199], so 05678 could be in block 2.

Parameters

<i>targetKey</i>	ZIP code string.
------------------	------------------

Returns

RBN, or -1 if targetKey is beyond all blocks.

3.6.2.3 read()

```
bool SimpleIndex::read (
    const string & filename)
```

Load the index from a text file into RAM.

Load the index from a text file.

Parameters

<i>filename</i>	Input file path.
-----------------	------------------

Returns

true on success.

Expects the format written by [write\(\)](#).

Parameters

<i>filename</i>	Path to read.
-----------------	---------------

Returns

true on success.

3.6.2.4 removeByKey()

```
void SimpleIndex::removeByKey (  
    const string & highestKey)
```

Remove the entry whose key equals *highestKey*.

Parameters

<i>highestKey</i>	The exact highest-key string to remove.
<i>highestKey</i>	Exact key string to remove.

3.6.2.5 removeByRBN()

```
void SimpleIndex::removeByRBN (  
    int rbn)
```

Remove the entry whose value equals *rbn*.

Remove the entry whose value (RBN) equals *rbn*.

Parameters

<i>rbn</i>	RBN of the block to remove from the index.
------------	--

Scans all entries (the index is small enough that linear scan is fine).

Parameters

<i>rbn</i>	RBN to remove.
------------	----------------

3.6.2.6 size()

```
int SimpleIndex::size () const
```

Return the number of entries in the index.

Return the number of index entries.

3.6.2.7 upsert()

```
void SimpleIndex::upsert (
    const string & highestKey,
    int rbn)
```

Insert or update an entry {highestKey → rbn}.

Insert or update the entry {highestKey → rbn}.

If an entry for this rbn already exists under a different key, the old entry is removed first.

Parameters

<i>highestKey</i>	Highest ZIP key in the block (e.g. "56310").
<i>rbn</i>	Relative Block Number of that block.

If the rbn already has an entry under a different key, the old entry is removed first (a block can only have one highest key at a time).

Parameters

<i>highestKey</i>	Highest ZIP key string in the block.
<i>rbn</i>	Relative Block Number.

3.6.2.8 write()

```
bool SimpleIndex::write (
    const string & filename) const
```

Write the index to a text file.

Parameters

<i>filename</i>	Output file path.
-----------------	-------------------

Returns

true on success.

File format: SIDX,1 <key> <rbn> ...

Parameters

<i>filename</i>	Path to write.
-----------------	----------------

Returns

true on success.

The documentation for this class was generated from the following files:

- [SimpleIndex.h](#)
- [SimpleIndex.cpp](#)

3.7 StateExtremes Struct Reference

Keeps the most extreme ZIP codes for one state.

Public Attributes

- int **easternmost**
- int **westernmost**
- int **northernmost**
- int **southernmost**
- double **minLong**
- double **maxLong**
- double **maxLat**
- double **minLat**

3.7.1 Detailed Description

Keeps the most extreme ZIP codes for one state.

The documentation for this struct was generated from the following file:

- [main.cpp](#)

3.8 ZipCodeBuffer Class Reference

Reads ZIP code records one at a time from a CSV file.

```
#include <ZipCodeBuffer.h>
```

Public Member Functions

- [ZipCodeBuffer](#) ()
Default constructor.
- **ZipCodeBuffer** (const string &csvFilename)
Constructor that opens a file immediately.
- **~ZipCodeBuffer** ()
Destructor.
- bool **open** (const string &csvFilename)
Opens a CSV file and processes its header row.
- void **close** ()
Closes the file and resets internal state.
- bool **isOpen** () const
Checks whether file is open.
- bool **readRecord** ([ZipCodeRecord](#) &record)
Reads one data record from the CSV file.
- bool **reset** ()
Resets file back to beginning and processes header row again.
- long **getRecordCount** () const
Returns how many records have been read.
- string **getFilename** () const
Returns current filename.
- vector< [ZipCodeRecord](#) > **gatherAllRecords** ()
Reads all records into a vector.

3.8.1 Detailed Description

Reads ZIP code records one at a time from a CSV file.

In Project 3 this is also used as the "record buffer": given a raw CSV string (extracted from a block by [BlockBuffer](#)), call `fromCSV()` on a [ZipCodeRecord](#) to unpack its fields.

3.8.2 Constructor & Destructor Documentation

3.8.2.1 ZipCodeBuffer()

```
ZipCodeBuffer::ZipCodeBuffer ()
```

Default constructor.

Initializes all member variables to safe starting values.

3.8.3 Member Function Documentation

3.8.3.1 gatherAllRecords()

```
vector< ZipCodeRecord > ZipCodeBuffer::gatherAllRecords ()
```

Reads all records into a vector.

This is kept for compatibility, but Project 2 should prefer reading one record at a time.

3.8.3.2 open()

```
bool ZipCodeBuffer::open (
    const string & csvFilename)
```

Opens a CSV file and processes its header row.

This function: 1) closes any previously open file 2) opens the requested file 3) reads the first line (header row) 4) finds the positions of ZIP, place, state, county, latitude, longitude

Parameters

<i>csvFilename</i>	CSV file path
--------------------	---------------

Returns

true on success, false on failure

3.8.3.3 readRecord()

```
bool ZipCodeBuffer::readRecord (
    ZipCodeRecord & record)
```

Reads one data record from the CSV file.

Parameters

<i>record</i>	Output record
---------------	---------------

Returns

true if record is read successfully, false on EOF or parse failure

3.8.3.4 reset()

```
bool ZipCodeBuffer::reset ()
```

Resets file back to beginning and processes header row again.

Returns

true if reset succeeds

The documentation for this class was generated from the following files:

- [ZipCodeBuffer.h](#)
- [ZipCodeBuffer.cpp](#)

3.9 ZipCodeRecord Struct Reference

Stores one complete ZIP code record.

```
#include <ZipCodeBuffer.h>
```

Public Member Functions

- **ZipCodeRecord** ()
Default constructor.
- **ZipCodeRecord** (int zip, const string &place, const string &st, const string &cnty, double lat, double lon)
Parameterized constructor.
- string **toCSV** () const
Serialize this record to a comma-separated string.
- bool **fromCSV** (const string &csv)
Populate this record from a comma-separated string.

Public Attributes

- int **zipCode**
5-digit ZIP code (primary key)
- string **placeName**
City / place name.
- string **state**
Two-letter state abbreviation.
- string **county**
County name.
- double **latitude**
Latitude coordinate.
- double **longitude**
Longitude coordinate.

3.9.1 Detailed Description

Stores one complete ZIP code record.

3.9.2 Member Function Documentation

3.9.2.1 fromCSV()

```
bool ZipCodeRecord::fromCSV (  
    const string & csv)
```

Populate this record from a comma-separated string.

Populate fields from a comma-separated string.

Parameters

csv	CSV text produced by toCSV()
-----	--

Returns

true on success

Expected order: ZipCode,PlaceName,State,County,Lat,Long

Parameters

csv	CSV text (e.g. from toCSV())
-----	---

Returns

true if all six fields were parsed successfully

3.9.2.2 toCSV()

```
string ZipCodeRecord::toCSV () const
```

Serialize this record to a comma-separated string.

Returns

CSV text (no newline), e.g. "56301,St Cloud,MN,Stearns,45.5579,-94.1632"

Field order matches the original CSV: ZipCode,PlaceName,State,County,Lat,Long

Returns

CSV text with no trailing newline

The documentation for this struct was generated from the following files:

- [ZipCodeBuffer.h](#)
- [ZipCodeBuffer.cpp](#)

Chapter 4

File Documentation

4.1 BlockBuffer.cpp File Reference

Implementation of the [BlockBuffer](#) class.

```
#include "BlockBuffer.h"
#include <sstream>
#include <iomanip>
#include <stdexcept>
#include <algorithm>
#include <iostream>
```

4.1.1 Detailed Description

Implementation of the [BlockBuffer](#) class.

See [BlockBuffer.h](#) for the block layout specification.

Author

Teagen Lee

Date

Spring 2026

4.2 BlockBuffer.h File Reference

Block buffer class for the blocked sequence set file.

```
#include <string>
#include <vector>
#include <fstream>
```

Classes

- class [BlockBuffer](#)

Reads and writes fixed-size blocks in a blocked sequence set file.

4.2.1 Detailed Description

Block buffer class for the blocked sequence set file.

A "blocked sequence set" stores length-indicated, comma-separated ZIP code records packed into fixed-size blocks on disk. Each block is exactly BLOCK_SIZE bytes. Blocks are addressed by their Relative Block Number (RBN), where RBN 0 is the header block (the data file header record) and RBN 1 is the first data/avail block.

– Block layout (active block) ————— Bytes 0–3 : record count (4-byte ASCII integer, space-padded, e.g. " 3") Bytes 4–11 : predecessor RBN (8-byte ASCII integer, e.g. "00000000") Bytes 12–19 : successor RBN (8-byte ASCII integer, e.g. "00000002") Bytes 20–end : packed records, each prefixed with a 4-byte ASCII length

– Block layout (avail block) ————— Bytes 0–3 : " 0" (record count == 0) Bytes 4–11 : "00000000" (predecessor unused) Bytes 12–19 : next-avail RBN (8-byte ASCII, or "00000000" for end-of-list) Bytes 20–end : blanks

– RBN encoding ————— RBN -1 (stored as "FFFFFFF") means "no link" (NULL) RBN 0 is reserved for the file header block. Data/avail blocks start at RBN 1.

Author

Teagen Lee

Date

Spring 2026

4.3 BlockBuffer.h

[Go to the documentation of this file.](#)

```
00001
00031
00032 #ifndef BLOCKBUFFER_H
00033 #define BLOCKBUFFER_H
00034
00035 #include <string>
00036 #include <vector>
00037 #include <fstream>
00038
00039 using namespace std;
00040
00041 // -----
00042 // Constants
00043 // -----
00044
00046 static const int DEFAULT_BLOCK_SIZE = 512;
00047
00049 static const int RBN_NULL = -1;
00050
00052 static const int BLOCK_COUNT_WIDTH = 4;
00053
00055 static const int BLOCK_RBN_WIDTH = 8;
00056
00058 static const int RECORD_LEN_WIDTH = 4;
00059
```



```

00061 static const int BLOCK_META_SIZE = BLOCK_COUNT_WIDTH + 2 * BLOCK_RBN_WIDTH;
00062 // = 4 + 8 + 8 = 20 bytes
00063
00064 // -----
00065 // BlockBuffer
00066 // -----
00067
00082 class BlockBuffer {
00083 public:
00084     // - Construction -----
00085
00090     explicit BlockBuffer(int blockSize = DEFAULT_BLOCK_SIZE);
00091
00092     // - Metadata accessors -----
00093
00094     int getBlockSize() const;
00095     int getRecordCount() const;
00096     int getPredRBN() const;
00097     int getSuccRBN() const;
00098     bool isAvail() const;
00099
00100     void setPredRBN(int rbn);
00101     void setSuccRBN(int rbn);
00102
00103     // - Record access -----
00104
00109     const vector<string>& getRecords() const;
00110
00120     bool addRecord(const string& csvRecord);
00121
00126     int freeBytes() const;
00127
00131     void clear();
00132
00133     // - Serialisation -----
00134
00145     string pack() const;
00146
00155     bool unpack(const string& raw);
00156
00157     // - Disk I/O -----
00158
00169     bool writeBlock(fstream& fs, int rbn) const;
00170
00181     bool readBlock(fstream& fs, int rbn);
00182
00192     long long offsetForRBN(int rbn) const;
00193
00194     // - Avail-block helpers -----
00195
00204     void makeAvail(int nextAvail = RBN_NULL);
00205
00206     // - Dump helpers -----
00207
00214     string highestKey() const;
00215
00221     string lowestKey() const;
00222
00223 private:
00224     int blockSize_;
00225     int predRBN_;
00226     int succRBN_;
00227     vector<string> records_;
00228
00230     string encodeInt(int value, int width) const;
00231
00234     int decodeInt(const string& s) const;
00235
00237     int maxPayload() const;
00238
00240     int usedPayload() const;
00241 };
00242
00243 #endif // BLOCKBUFFER_H

```

4.4 BlockHeaderBuffer.cpp File Reference

Implementation of [BlockHeaderBuffer](#).

```

#include "BlockHeaderBuffer.h"
#include "BlockBuffer.h"

```

```
#include <sstream>
#include <iomanip>
#include <iostream>
#include <cctype>
#include <stdexcept>
```

4.4.1 Detailed Description

Implementation of [BlockHeaderBuffer](#).

Author

Teagen Lee

Date

Spring 2026

4.5 BlockHeaderBuffer.h File Reference

Header record class for the blocked sequence set data file.

```
#include <string>
#include <vector>
#include <fstream>
```

Classes

- struct [BFieldDescriptor](#)
Stores the name and type format for one field in the blocked file.
- struct [BlockFileHeader](#)
All header fields for a blocked sequence set data file.
- class [BlockHeaderBuffer](#)
Reads and writes the header record for a blocked sequence set file.

4.5.1 Detailed Description

Header record class for the blocked sequence set data file.

Extends the Project 2 header concept to include all metadata required by the assignment for a blocked sequence set file:

- file structure type
- version
- header record size

- bytes used for each record-size integer
- size format type (ASCII)
- block size
- minimum block capacity percentage
- index file name
- index file schema
- record count
- block count
- field count + per-field name/type
- primary key field index
- avail-list head RBN
- sequence-set list head RBN
- stale flag

The header is stored in RBN 0 (the first block of the file) as a length-indicated, comma-separated record padded with spaces to blockSize.

Author

Teagen Lee

Date

Spring 2026

4.6 BlockHeaderBuffer.h

[Go to the documentation of this file.](#)

```

00001
00031
00032 #ifndef BLOCKHEADERBUFFER_H
00033 #define BLOCKHEADERBUFFER_H
00034
00035 #include <string>
00036 #include <vector>
00037 #include <fstream>
00038
00039 using namespace std;
00040
00041 // -----
00042 // Supporting structures
00043 // -----
00044
00049 struct BFieldDescriptor {
00050     string name;
00051     string format;
00052 };
00053
00058 struct BlockFileHeader {
00059     string fileType;
00060     int version;
00061     int headerSizeBytes;
00062     int recordSizeBytes;
00063     string sizeFormatType;
00064     int blockSize;

```

```

00065     int     minBlockCapacityPct;
00066     string  indexFileName;
00067     string  indexSchema;
00068     long long recordCount;
00069     int     blockCount;
00070     int     fieldCount;
00071     vector<BFieldDescriptor> fields;
00072     int     primaryKeyIndex;
00073     int     availHeadRBN;
00074     int     seqSetHeadRBN;
00075     bool    staleFlag;
00076 };
00077
00078 // -----
00079 // BlockHeaderBuffer
00080 // -----
00081
00082 class BlockHeaderBuffer {
00083 public:
00084     // - Construction -----
00085
00086     explicit BlockHeaderBuffer(int blockSize = 512);
00087
00088     // - Build -----
00089
00090     void buildDefault(const string& indexFileName,
00091                     int     blockSize,
00092                     long long recordCount,
00093                     int     blockCount,
00094                     int     availHead,
00095                     int     seqHead);
00096
00097     // - I/O -----
00098
00099     bool write(fstream& fs) const;
00100
00101     bool read(fstream& fs);
00102
00103     // - Accessors/Mutators -----
00104
00105     const BlockFileHeader& getHeader() const;
00106     BlockFileHeader&      getHeader();
00107
00108     void setRecordCount(long long count);
00109     void setBlockCount(int count);
00110     void setAvailHead(int rbn);
00111     void setSeqSetHead(int rbn);
00112     void setStale(bool s);
00113
00114     // - Display -----
00115
00116     void print() const;
00117
00118 private:
00119     BlockFileHeader hdr_;
00120     int             blockSize_;
00121
00122     string serialize() const;
00123
00124     bool deserialize(const string& s);
00125 };
00126
00127 #endif // BLOCKHEADERBUFFER_H

```

4.7 main.cpp File Reference

Zip Code Group Project 3.0 — Blocked Sequence Set main controller.

```

#include "SequenceSet.h"
#include "ZipCodeBuffer.h"
#include "SimpleIndex.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

```

```
#include <map>
#include <iomanip>
#include <limits>
#include <algorithm>
```

Classes

- struct [StateExtremes](#)
Keeps the most extreme ZIP codes for one state.

Functions

- int **main** (int argc, char *argv[])
Program entry point. Dispatches to the appropriate mode.

4.7.1 Detailed Description

Zip Code Group Project 3.0 — Blocked Sequence Set main controller.

This is a multi-mode program. Each mode is selected by the first command line argument.

	MODES
1) Create a blocked sequence set from a sorted CSV: <code>./zip3 -create <sorted.csv> <out.bss> <out.sidx> [blockSize]</code>	
2) Dump (physical then logical) an existing blocked sequence set: <code>./zip3 -dump <data.bss> <data.sidx> [blockSize]</code>	
3) Dump just the simple index: <code>./zip3 -dump-index <data.bss> <data.sidx> [blockSize]</code>	
4) Search for ZIP codes (loads index into RAM, never loads whole data file): <code>./zip3 -search <data.bss> <data.sidx> -Z56301 -Z99546 -Z00001 [blockSize]</code>	
5) Project-1-style sequential analysis (state extremes): <code>./zip3 -analyze <data.bss> <data.sidx> [blockSize]</code>	
6) Add records from a file of CSV lines: <code>./zip3 -add <data.bss> <data.sidx> <records_to_add.csv> [blockSize]</code>	
7) Delete records whose keys are listed in a file (one ZIP per line): <code>./zip3 -delete <data.bss> <data.sidx> <keys_to_delete.txt> [blockSize]</code>	
8) Print the file header: <code>./zip3 -header <data.bss> <data.sidx> [blockSize]</code>	

	NOTES
• The blocked sequence set file (.bss) stores length-indicated, comma-separated records in fixed-size blocks.	
• The simple index file (.sidx) stores {highestKey, RBN} pairs and is loaded entirely into RAM during search / add / delete.	
• Block splits, merges, and redistributions are logged to stdout.	
• All program variables that can vary are set by command line or metadata.	

Author

Teagen Lee

Date

Spring 2026

4.8 SequenceSet.cpp File Reference

Implementation of the [SequenceSet](#) class.

```
#include "SequenceSet.h"
#include <iostream>
#include <iomanip>
#include <sstream>
#include <algorithm>
#include <stdexcept>
#include <functional>
```

4.8.1 Detailed Description

Implementation of the [SequenceSet](#) class.

Author

Teagen Lee

Date

Spring 2026

4.9 SequenceSet.h File Reference

High-level manager for the blocked sequence set file.

```
#include "BlockBuffer.h"
#include "BlockHeaderBuffer.h"
#include "SimpleIndex.h"
#include "ZipCodeBuffer.h"
#include <string>
#include <fstream>
#include <vector>
#include <functional>
```

Classes

- class [SequenceSet](#)
Manages a blocked sequence set file for ZIP code records.

4.9.1 Detailed Description

High-level manager for the blocked sequence set file.

[SequenceSet](#) coordinates:

- Creating a blocked sequence set from a sorted CSV file.
- Sequential reading (for Project 1 style analysis).
- Physical and logical dump methods.
- Index-based record search.
- Record insertion (with block splitting).
- Record deletion (with redistribution / merging).
- Avail-list management.

It uses:

- [BlockBuffer](#) to pack/unpack individual blocks.
- [BlockHeaderBuffer](#) to read/write the file header (RBN 0).
- [SimpleIndex](#) to maintain the in-RAM {highestKey, RBN} index.
- [ZipCodeRecord](#) as the typed record object.

Author

Teagen Lee

Date

Spring 2026

4.10 SequenceSet.h

[Go to the documentation of this file.](#)

```
00001
00023
00024 #ifndef SEQUENCESET_H
00025 #define SEQUENCESET_H
00026
00027 #include "BlockBuffer.h"
00028 #include "BlockHeaderBuffer.h"
00029 #include "SimpleIndex.h"
00030 #include "ZipCodeBuffer.h"
00031
00032 #include <string>
00033 #include <fstream>
00034 #include <vector>
00035 #include <functional>
00036
00037 using namespace std;
00038
00043 class SequenceSet {
00044 public:
00045     // - Construction / Open / Close -----
00046
```

```

00051     explicit SequenceSet(int blockSize = DEFAULT_BLOCK_SIZE);
00052
00053     ~SequenceSet();
00054
00073     bool create(const string& csvFile,
00074                const string& dataFile,
00075                const string& indexFile);
00076
00086     bool open(const string& dataFile, const string& indexFile);
00087
00091     void close();
00092
00096     bool isOpen() const;
00097
00098     // - Sequential read (Project 1 style) -----
00099
00108     void scanAll(function<void(const ZipCodeRecord&)> visitor) const;
00109
00110     // - Dump methods -----
00111
00122     void dumpPhysical() const;
00123
00129     void dumpLogical() const;
00130
00131     // - Index dump -----
00132
00136     void dumpIndex() const;
00137
00138     // - Search -----
00139
00150     bool search(const string& zipKey, ZipCodeRecord& result);
00151
00152     // - Insert -----
00153
00170     bool insert(const ZipCodeRecord& rec);
00171
00172     // - Delete -----
00173
00189     bool remove(const string& zipKey);
00190
00191     // - Header accessors -----
00192
00196     void printHeader() const;
00197
00198 private:
00199     // - Member data -----
00200
00201     int          blockSize_;
00202     mutable fstream fs_;
00203     string        dataFile_;
00204     string        indexFile_;
00205     BlockHeaderBuffer headerBuf_;
00206     SimpleIndex    index_;
00207     bool          isOpen_;
00208
00209     // - Private helpers -----
00210
00215     int appendBlock();
00216
00225     int getAvailBlock();
00226
00235     void releaseBlock(int rbn);
00236
00240     void flushHeader();
00241
00248     bool readBlock(int rbn, BlockBuffer& bb) const;
00249
00256     bool writeBlock(int rbn, BlockBuffer& bb);
00257
00263     static string keyOf(const string& csvRecord);
00264
00277     int minRecords(const BlockBuffer& bb) const;
00278
00283     int findTailRBN() const;
00284
00290     static void insertSorted(vector<string>& records, const string& csv);
00291
00298     static bool removeByKey(vector<string>& records, const string& key);
00299
00311     static void printBlockLine(int rbn, const BlockBuffer& bb);
00312 };
00313
00314 #endif // SEQUENCESET_H

```


4.11 SimpleIndex.cpp File Reference

Implementation of [SimpleIndex](#).

```
#include "SimpleIndex.h"
#include <iostream>
#include <iomanip>
#include <sstream>
#include <algorithm>
```

4.11.1 Detailed Description

Implementation of [SimpleIndex](#).

Author

Teagen Lee

Date

Spring 2026

4.12 SimpleIndex.h File Reference

Simple primary key index for the blocked sequence set.

```
#include <string>
#include <map>
#include <vector>
#include <fstream>
```

Classes

- class [SimpleIndex](#)
In-RAM representation of the simple primary key index.

4.12.1 Detailed Description

Simple primary key index for the blocked sequence set.

This index stores ordered pairs {highest key in block, RBN} as described in Folk Figure 10.3. The index is small enough to load entirely into RAM.

On disk the index file has this format: Line 0: SIDX,1 (file type tag + version) Line N: <key> <rbn> (one entry per active block)

Example: SIDX,1 01005 1 02139 2 56301 3

Author

Teagen Lee

Date

Spring 2026

4.13 SimpleIndex.h

[Go to the documentation of this file.](#)

```

00001
00021
00022 #ifndef SIMPLEINDEX_H
00023 #define SIMPLEINDEX_H
00024
00025 #include <string>
00026 #include <map>
00027 #include <vector>
00028 #include <fstream>
00029
00030 using namespace std;
00031
00032 // -----
00033 // SimpleIndex
00034 // -----
00035
00050 class SimpleIndex {
00051 public:
00052     SimpleIndex();
00053
00054     // - Build / Modify -----
00055
00065     void upsert(const string& highestKey, int rbn);
00066
00072     void removeByRBN(int rbn);
00073
00079     void removeByKey(const string& highestKey);
00080
00081     // - Search -----
00082
00092     int findRBN(const string& targetKey) const;
00093
00094     // - I/O -----
00095
00102     bool write(const string& filename) const;
00103
00110     bool read(const string& filename);
00111
00112     // - Accessors -----
00113
00117     int size() const;
00118
00122     vector<pair<string, int>> entries() const;
00123
00124     // - Display -----
00125
00129     void dump() const;
00130
00131 private:
00133     map<string, int> index_;
00134 };
00135
00136 #endif // SIMPLEINDEX_H

```

4.14 ZipCodeBuffer.cpp File Reference

Implementation of the [ZipCodeBuffer](#) class.

```

#include "ZipCodeBuffer.h"
#include <algorithm>
#include <cctype>
#include <iomanip>
#include <stdexcept>

```

4.14.1 Detailed Description

Implementation of the [ZipCodeBuffer](#) class.

This version supports CSV files even when the columns are re-ordered. It reads the header row first, remembers the needed column positions, and then parses data rows using those positions.

4.15 ZipCodeBuffer.h File Reference

Record buffer class for reading ZIP code records from CSV files.

```
#include <string>
#include <fstream>
#include <sstream>
#include <vector>
```

Classes

- struct [ZipCodeRecord](#)
Stores one complete ZIP code record.
- class [ZipCodeBuffer](#)
Reads ZIP code records one at a time from a CSV file.

4.15.1 Detailed Description

Record buffer class for reading ZIP code records from CSV files.

Carried forward from Project 2 with no functional changes. In Project 3 this class serves as the "record buffer" layer: [BlockBuffer](#) → unpacks a raw record string from a block [ZipCodeBuffer](#) (record buffer role) → parses that string into a [ZipCodeRecord](#)

Author

Teagen Lee

Date

Spring 2026

4.16 ZipCodeBuffer.h

[Go to the documentation of this file.](#)

```
00001
00013
00014 #ifndef ZIPCODEBUFFER_H
00015 #define ZIPCODEBUFFER_H
00016
00017 #include <string>
00018 #include <fstream>
00019 #include <sstream>
00020 #include <vector>
00021
00022 using namespace std;
00023
00024 // =====
00025 // ZipCodeRecord
00026 // =====
00027
00032 struct ZipCodeRecord {
00033     int    zipCode;
00034     string placeName;
00035     string state;
00036     string county;
```

```

00037     double latitude;
00038     double longitude;
00039
00040     ZipCodeRecord();
00041     ZipCodeRecord(int zip, const string& place, const string& st,
00042                   const string& cnty, double lat, double lon);
00043
00044     string toCSV() const;
00045
00046     bool fromCSV(const string& csv);
00047 };
00048
00049 // =====
00050 // ZipCodeBuffer
00051 // =====
00052
00053 class ZipCodeBuffer {
00054 public:
00055     ZipCodeBuffer();
00056     explicit ZipCodeBuffer(const string& csvFilename);
00057     ~ZipCodeBuffer();
00058
00059     bool open(const string& csvFilename);
00060     void close();
00061     bool isOpen() const;
00062     bool readRecord(ZipCodeRecord& record);
00063     bool reset();
00064     long getRecordCount() const;
00065     string getFilename() const;
00066
00067     vector<ZipCodeRecord> gatherAllRecords();
00068 private:
00069     ifstream fileStream;
00070     string filename;
00071     bool headerSkipped;
00072     long recordCount;
00073
00074     int colZip, colPlace, colState, colCounty, colLat, colLong;
00075
00076     bool parseLine(const string& line, ZipCodeRecord& record);
00077     bool parseHeader(const string& headerLine);
00078     vector<string> splitCSV(const string& line);
00079     string trim(const string& str);
00080 };
00081
00082 #endif // ZIPCODEBUFFER_H

```